

Algorithms COMP.5030 — Intro Lecture

(structured from NuMarkdown-8B-Thinking OCR, corrected)

Prof. Hugo A. Akitaya

1 Course Introduction

Slide 1: Title

Algorithms COMP.5030

Prof. Hugo A. Akitaya

Slide 2: What is Computer Science?

Map of Computer Science infographic.

https://www.youtube.com/watch?v=SzJ46YA_RaA

Slide 3: Why study algorithms?

- Job interviews.
- Solving (new/old) problems.
- Being a better coder.
- Understanding what can or cannot be done, and why.

(Image: “It’s dangerous to go alone! Take this.” — *The Legend of Zelda*.)

Slide 4: Algorithms?

Comic panels illustrating the struggle with algorithms (xkcd).

<https://xkcd.com/1831/>

Slide 5: Prerequisites

- Discrete Math:
 - Proof techniques
 - Basic graph theory
- COMP.4040:
 - Fundamental algorithms

- Asymptotic analysis
- Basic data types/structures
- Writing pseudocode

(Image: cover of *Introduction to Algorithms* by Cormen et al.)

Slide 6: Blackboard

- Syllabus
- Homework
- Attendance
- Slido

Slide 7: Poll: Why 5030?

Slido poll Why 5030? (open-ended question)
--

2 Review: The Maximum-Profit Problem

Slide 8: Problem statement (CLRS 4.1)

Problem: You are allowed to buy and sell a stock exactly once. You know the future and have a graph of future stock prices. **Maximize your profit.**

Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97

Slide 9: Understanding the problem

- **Input:** Array of integers $A[1..n]$.
- **Output:** Indices i and j (with $i \leq j$) such that $A[j] - A[i]$ is maximized.

(Line graph of the daily prices from Slide 8.)

Slide 10: Poll: What is the optimal solution?

Slido poll What is the optimal solution? (multiple choice)
--

Slide 11: Poll: Which of these solutions is better?

Slido poll Which of these solutions is better? (multiple choice)
--

Slide 12: A little more math

Profit function:

$$p : \{(i, j) \mid i, j \in [0..n-1], i \leq j\} \rightarrow \mathbb{R}, \quad p(i, j) = A[j] - A[i].$$

- $p(1, 3) = ?$
- Optimal solution: $\arg \max_{(i, j)} p(i, j)$.

Slide 13: Brute-force solution (pseudocode review)

- Try all possible outputs and see which is the best answer.
- Idea: try every i and j with $i < j$.
- Pseudocode review: functional, control-flow keywords in **bold**, indentation matters.

Max-Profit-Brute-Force($A[1..n]$)

```
max, left, right = 0
for i = 1 to n - 1
    for j = i + 1 to n
        if  $A[j] - A[i] > \textit{max}$  (pair discussion)
            max =  $A[j] - A[i]$ 
            left = i
            right = j
return (left, right)
```

(Small example: days 0–4 with prices 10, 11, 7, 10, 6.)

Slide 14: Brute-force analysis

- **Correctness**
 - Easy: we check all possibilities, by exhaustion.
 - Loop invariant: at each iteration, *max* stores the best solution so far.
- **Time complexity:** (see poll on Slide 15; $\Theta(n^2)$ pairs)

Max-Profit-Brute-Force($A[1..n]$)

```
max, left, right = 0
for i = 1 to n - 1
    for j = i + 1 to n
        if  $A[j] - A[i] > \textit{max}$ 
            max =  $A[j] - A[i]$ 
            left = i
            right = j
return (left, right)
```

Slide 15: Poll: time complexity of brute force

Slido poll

What is the time complexity of the brute-force algorithm? (multiple choice)

3 Asymptotic Notation

Slide 16: Building math vocabulary: Big-O

The main idea: Big-O is a way to talk about how fast a function (often an algorithm's running time) grows when the input size n gets very large.

- We ignore small details and constants, and only care about the *overall growth trend* for big n .
- Saying $f(n) = O(g(n))$ means: **for large enough n , $f(n)$ does not grow faster than $g(n)$ (up to a constant multiplier).**
- Goal: compare functions asymptotically.
- $O(g(n))$ is a *set* of functions.

What the definition is saying:

$$f(n) = O(g(n)) \iff \exists c > 0, n_0 \text{ such that } |f(n)| \leq c \cdot g(n) \quad \forall n \geq n_0.$$

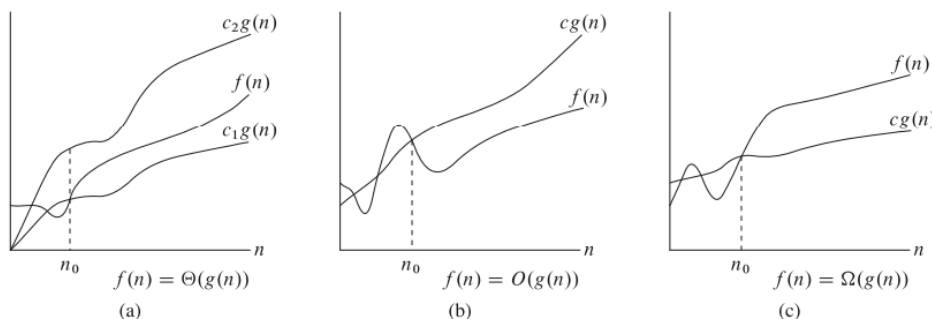
Simple translation:

- $\exists c > 0$: there is some constant number c .
- $\exists n_0$: there is some point n_0 after which we start caring.
- For every $n \geq n_0$: **after that point**.
- $f(n)$ is always **at most** $c \cdot g(n)$.

So $g(n)$ is like an **upper bound** (a “ceiling”) for $f(n)$, ignoring constant factors.

What the pictures mean:

- (a) $f(n) = \Theta(g(n))$: $f(n)$ is sandwiched between two constant multiples of $g(n) \Rightarrow$ same growth rate (tight bound).
- (b) $f(n) = O(g(n))$: $f(n)$ stays below some multiple of $g(n)$ after $n_0 \Rightarrow$ upper bound.
- (c) $f(n) = \Omega(g(n))$: $f(n)$ stays above some multiple of $g(n)$ after $n_0 \Rightarrow$ lower bound.



Quick example: if $f(n) = 3n+10$, then $f(n) = O(n)$, because for large n it grows like n (constants don't matter).

Slide 17: Poll: which sets contain this function?

Slido poll

Which of these sets contain $n^2 + n^{4.5} + \sqrt{n} + \log n$? (multiple choice)

Worked solution. We want to know which Big-O / Θ / Ω “sets” include the function

$$f(n) = n^2 + n^{4.5} + \sqrt{n} + \log n.$$

1) Find the dominant term. As n gets large, the term that grows fastest dominates everything else. Compare the powers:

- $n^{4.5}$ grows faster than n^2 .
- $n^{4.5}$ grows much faster than $\sqrt{n} = n^{0.5}$.
- $n^{4.5}$ grows much faster than $\log n$.

So for large n , $f(n) \approx n^{4.5}$. More formally: $f(n) = \Theta(n^{4.5})$.

2) What sets contain it? Because $f(n) = \Theta(n^{4.5})$, all of these are true:

- **Big-Theta:** $f(n) \in \Theta(n^{4.5})$.
- **Big-O (upper bounds):** $f(n) \in O(n^{4.5}), O(n^5), O(n^{10}), O(2^n)$, etc. (any function that grows *at least as fast* as $n^{4.5}$ can be an upper bound).
- **Big-Omega (lower bounds):** $f(n) \in \Omega(n^{4.5}), \Omega(n^2), \Omega(\sqrt{n}), \Omega(\log n), \Omega(1)$ (any function that grows *no faster* than $n^{4.5}$ can be a lower bound).

3) What it is NOT in:

- Not in $O(n^2)$ (because $n^{4.5}$ eventually dwarfs n^2).
- Not in $\Theta(n^2), \Theta(n^5)$, etc. (Theta must match the growth rate).

Slide 18: Building math vocabulary: the full family

Big picture: think of $f(n)$ as the “real cost” (time, steps, etc.) and $g(n)$ as a “reference growth”. We only care about what happens **after some point** n_0 (when n is big).

1) Big-O: $f(n) = O(g(n))$ (**upper bound**). Meaning: f grows *no faster* than g (up to a constant). After n_0 , $f(n) \leq c \cdot g(n)$ for some constant c . Plain English: “ $g(n)$ is a *ceiling* for $f(n)$ (ignoring constant factors).” Example: $3n + 10 = O(n)$.

2) Big-Omega: $f(n) = \Omega(g(n))$ (**lower bound**). Meaning: f grows *at least as fast* as g (up to a constant). After n_0 , $f(n) \geq c \cdot g(n)$. Plain English: “ $g(n)$ is a *floor* for $f(n)$.” Example: $3n + 10 = \Omega(n)$.

3) Big-Theta: $f(n) = \Theta(g(n))$ (**tight bound**). Meaning: *same growth rate*. It is both $O(g(n))$ (not faster) and $\Omega(g(n))$ (not slower), so f is “sandwiched” between two multiples of g . Example: $3n + 10 = \Theta(n)$. That is why $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$ (intersection = both true).

4) little-o: $f(n) = o(g(n))$ (**strictly smaller**). Meaning: f grows *slower* than g (not just “not faster”). In fact, no matter how small a constant c you pick, eventually $f(n) < c \cdot g(n)$. Example: $n = o(n^2)$.

5) little-omega: $f(n) = \omega(g(n))$ (**strictly larger**). Meaning: f grows *faster* than g . No matter how big a constant c you pick, eventually $f(n) > c \cdot g(n)$. Example: $n^2 = \omega(n)$.

Formal definitions:

- $f(n) = O(g(n)) \iff \exists c > 0, n_0$ such that $|f(n)| \leq c \cdot g(n) \quad \forall n \geq n_0$.
- $f(n) = \Omega(g(n)) \iff \exists c > 0, n_0$ such that $|f(n)| \geq c \cdot g(n) \quad \forall n \geq n_0$.
- $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$.
- $f(n) = o(g(n)) \iff \forall c > 0, \exists n_0$ such that $|f(n)| < c \cdot g(n) \quad \forall n \geq n_0$. (strictly slower growth)
- $f(n) = \omega(g(n)) \iff \forall c > 0, \exists n_0$ such that $|f(n)| > c \cdot g(n) \quad \forall n \geq n_0$. (strictly faster growth)

Memory shortcut: O = “at most”, Ω = “at least”, Θ = “same”, o = “strictly less”, ω = “strictly more”.

Slide 19: Poll: $f(n) = \log^2 n$

Slido poll
Select all that apply: $f(n) = \log^2 n$

Slide 20: Poll: comprehension check

Slido poll
How well do you understand Big-O? (rating)

4 Divide and Conquer

Slide 21: Back to the Max-Profit problem

Divide and Conquer approach:

- Define subproblems (smaller instances) that can be solved independently.
- **Optimal substructure:** an optimal solution contains optimal solutions to subproblems.
- We can build an optimal solution for the original problem using optimal solutions to subproblems.
- Hard part: defining the subproblems.

Slide 22: Subproblems: split A in half

Three cases:

1. OPT is entirely in the left subproblem: $A[low..mid]$
2. OPT is entirely in the right subproblem: $A[mid + 1..high]$
3. OPT crosses the midpoint (pair discussion)

(Diagram: array segment with indices low , mid , $mid + 1$, $high$; cases 1–2 entirely on one side of the split, case 3 crossing the midpoint.)

Slide 23: Case 3: structure of the crossing OPT

- If an oracle told you OPT crosses the midpoint, what can you say about $A[i]$ and $A[j]$?
- Can $A[i]$ be greater than $\min(A[low..mid])$? (No.)
- Then OPT is

$$\left(\arg \min_{i \in [low..mid]} A[i], \arg \max_{j \in [mid+1..high]} A[j] \right).$$

(Small example: days 0–4 with prices 10, 11, 7, 10, 6.)

Slide 24: Full example

Divide and Conquer approach:

- Divide subproblems.
- Compute solutions to subproblems.
- Compute the crossing solution.
- Return the best of the 3 cases.

(Small example: days 0–4 with prices 10, 11, 7, 10, 6.)

Slide 25: Correctness + pseudocode

Divide and Conquer:

- Show optimal substructure:
 - Case analysis + contradiction
 - Induction $\sim$ Recursion

Max-Profit-D&C(A , low , $high$)

```

if  $low == high$ 
    return ( $low$ ,  $high$ )
else  $mid = \lfloor (low + high)/2 \rfloor$ 
    ( $l\_i$ ,  $l\_j$ ) = Max-Profit-D&C(_____)
    ( $r\_i$ ,  $r\_j$ ) = Max-Profit-D&C(_____)
    ( $c\_i$ ,  $c\_j$ ) = ( Find-min-index(_____), Find-max-index(_____) )
    return best ( $x\_i$ ,  $x\_j$ ) with  $x \in \{l, r, c\}$ 

```

Slide 26: Time analysis

Recurrence describing $T(n)$:

Max-Profit-D&C(A , low , $high$)

```

if  $low == high$ 
    return ( $low$ ,  $high$ )

```

```

else mid = ⌊ (low + high)/2 ⌋
    (l_i, l_j) = Max-Profit-D&C(A, low, mid)
    (r_i, r_j) = Max-Profit-D&C(A, mid+1, high)
    (c_i, c_j) = ( Find-min-index(A, low, mid), Find-max-index(A, mid+1, high))
return best (x_i, x_j) with x ∈ {l, r, c}

```

Plain-English walkthrough (D&C for Max-Profit). The algorithm is a **Divide-and-Conquer** solution for the Max-Profit problem.

1) The Max-Profit problem. Given prices over days (e.g., 10, 11, 7, 10, 6 on days 0–4), buy at a low price and sell later at a high price to maximize profit. Example: buy on day 2 for 7, sell on day 3 for 10, profit = 10 – 7 = 3. The goal is to find the pair (buy day, sell day) with the largest positive difference.

2) Divide the array in half. Split around the middle:

```

      mid
      ↓
[10, 11, 7] | [10, 6]
    LEFT      RIGHT

```

The best solution is in exactly one of three cases:

- **Case 1:** entirely on the left — find the best buy/sell pair using only the left side.
- **Case 2:** entirely on the right — find the best pair using only the right side.
- **Case 3:** crosses the middle — the purchase happens on the **left**, the sale on the **right**, because the transaction crosses the midpoint.

3) Finding the crossing solution. For a crossing solution:

- **Buy side:** take the *minimum* price on the left. Left side 10, 11, 7: minimum = 7, so buy = 7.
- **Sell side:** take the *maximum* price on the right. Right side 10, 6: maximum = 10, so sell = 10.
- Crossing profit = 10 – 7 = 3.

4) Why minimum on the left and maximum on the right? Suppose the optimal crossing solution buys at position i and sells at j . If $A[i]$ is not the smallest value on the left, some $A[k] < A[i]$ exists and we could buy at k instead, giving at least as much profit. Similarly, if $A[j]$ is not the largest value on the right, we could sell higher and earn more. Therefore, for a crossing solution: **buy at the minimum price on the left and sell at the maximum price on the right.** That is why the algorithm computes $\min(A[\text{low}..\text{mid}])$ and $\max(A[\text{mid} + 1..\text{high}])$ and the profit $\text{max_right} - \text{min_left}$.

5) Three answers at every recursive step.

```

left  = solve(left half)           best profit completely in LEFT
right = solve(right half)          best profit completely in RIGHT
cross = max(right side) - min(left side)  best profit starting LEFT, ending RIGHT

```

return best of: left, right, cross

6) What the recursion does. Keep splitting the array; at a single element there is nothing to buy and sell within one day, so that is the **base case**. Then the algorithm works its way back up, comparing the three possibilities at every level.

7) The pseudocode, without the notation. It is basically:

Split → **solve left** → **solve right** → **solve crossing** → **pick the best**.

8) Full example. For prices 10, 11, 7, 10, 6 (days 0–4) the best transaction is buy day 2 at \$7, sell day 3 at \$10, profit \$3. The algorithm discovers this by comparing LEFT, RIGHT, and CROSS at each level and choosing the largest profit.

9) Why is this useful? Brute force compares every buy/sell pair: $O(n^2)$. Divide and Conquer splits the problem in half and recursively solves it, with recurrence $T(n) = 2T(n/2) + O(n)$ (the $O(n)$ comes from finding the minimum on the left and the maximum on the right). By the Master Theorem, $T(n) = O(n \log n)$:

Brute force: $O(n^2)$ vs. Divide & Conquer: $O(n \log n)$.

The one sentence to memorize: *Divide the array in half, find the best solution on the left, the best on the right, and the best solution crossing the middle; then return the best of the three. And for the crossing case specifically: buy at the minimum price on the left and sell at the maximum price on the right.*

Python implementation.

```
def max_profit_dc(A, low, high):
    # Base case: only one day
    if low == high:
        return (low, high)

    # Find the middle
    mid = (low + high) // 2

    # Case 1: best solution entirely in the left half
    left = max_profit_dc(A, low, mid)

    # Case 2: best solution entirely in the right half
    right = max_profit_dc(A, mid + 1, high)

    # Case 3: best solution crosses the midpoint
    min_index = low
    for i in range(low, mid + 1):
        if A[i] < A[min_index]:
            min_index = i

    max_index = mid + 1
    for j in range(mid + 1, high + 1):
        if A[j] > A[max_index]:
```

```

    max_index = j

    cross = (min_index, max_index)

    # Choose the best of the three
    left_profit = A[left[1]] - A[left[0]]
    right_profit = A[right[1]] - A[right[0]]
    cross_profit = A[cross[1]] - A[cross[0]]

    if left_profit >= right_profit and left_profit >= cross_profit:
        return left
    elif right_profit >= left_profit and right_profit >= cross_profit:
        return right
    else:
        return cross

```

Slide 27: Poll: solving the recurrence

Slido poll

What is the solution of the recurrence? (multiple choice)

Worked solution. You have a **recurrence** that describes how long an algorithm takes:

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

- $T(n)$ = time for input size n .
- $2 \cdot T(n/2)$ = we solve 2 smaller problems, each half the size.
- $+ \Theta(n)$ = extra work to combine results (like merging).

What is the solution? If you *expand* this pattern:

- Level 0: work = $\Theta(n)$.
- Level 1: 2 problems of size $n/2$, work = $2 \cdot \Theta(n/2) = \Theta(n)$.
- Level 2: 4 problems of size $n/4$, work = $4 \cdot \Theta(n/4) = \Theta(n)$.

Each **level** does $\Theta(n)$ work. How many levels? You keep dividing n by 2 until you reach 1, so that is $\log_2 n$ levels.

$$\text{Total work} = \Theta(n) \cdot \log_2 n = \Theta(n \log n).$$

Two methods to find this:

- **Recursion tree:** draw the tree of recursive calls; notice each level costs $\Theta(n)$; count levels $\rightarrow \log n$; gives the answer $\Theta(n \log n)$.
- **Substitution method:** *guess* the answer (use the recursion tree to guess), then *prove* it with induction (like a math proof). In this course, a super formal proof is not required.

Final simple answer:

$$T(n) = \Theta(n \log n) \quad (\text{like Merge Sort}).$$

That's it — you split in half, do linear work at each level, and there are $\log n$ levels.

5 Solving Recurrences

Slide 28: Recursion tree method

$$T(n) \leq \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2 \cdot T(n/2) + \Theta(n) & \text{otherwise.} \end{cases}$$

- Relies on noticing a pattern.
- For a formal proof, one must show that the pattern holds for all levels.
- We will not be very formal with this method in this course.

Slide 29: Substitution method

- Basically, induction using the definition of Big-O/Omega.
- We need to guess the answer (use the recursion tree first, for example).

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

Detailed walkthrough: recursion tree and substitution. 1) **What does the recurrence mean?** Think of it as: a **problem of size n** is split into **2 smaller problems**, each of size $n/2$, and we spend $\Theta(n)$ extra work to combine them:

$$\begin{array}{c} T(n) \\ / \quad \backslash \\ T(n/2) \quad T(n/2) \\ / \quad \backslash \quad \dots \quad \backslash \quad / \\ T(n/4) \quad \dots \quad T(n/4) \end{array}$$

Three important pieces: **2** = number of subproblems, $n/2$ = size of each subproblem, $\Theta(n)$ = work done outside the recursive calls.

2) Recursion tree method. The idea: *draw the tree and add up the work at each level.*

- **Level 0:** 1 problem: $\Theta(n)$.
- **Level 1:** 2 problems, each doing $\Theta(n/2)$: total $2 \cdot \Theta(n/2) = \Theta(n)$.
- **Level 2:** 4 problems, each doing $\Theta(n/4)$: total $4 \cdot \Theta(n/4) = \Theta(n)$.

$$\begin{array}{lcl} \text{Level 0:} & \Theta(n) & \\ & | & \\ \text{Level 1:} & \Theta(n/2) + \Theta(n/2) = \Theta(n) & \\ & / & \text{Level 2: } 4 \times \Theta(n/4) = \Theta(n) \\ & \dots & \end{array}$$

Every level costs $\Theta(n)$.

How many levels? Every time we divide n by 2, $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$, and we stop when we reach 1:

$$\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n.$$

So there are approximately $\log n$ levels. Since each level costs $\Theta(n)$:

$$T(n) = \Theta(n) \times \Theta(\log n) \implies \boxed{T(n) = \Theta(n \log n)}.$$

Recursion tree shortcut: if *each level has the same cost* and there are $\log n$ levels, the answer is $n \log n$.

3) Substitution method. The slide says: “basically, induction using the definition of Big-O/Omega.” The idea: *guess the answer first, then prove that your guess is correct.* From the recursion tree we already guessed $T(n) = \Theta(n \log n)$. Now we prove it. Suppose $T(n) \leq cn \log n$ for some constant c . Substitute this into the recurrence (ignoring constants for simplicity):

$$T(n) \leq 2 \left[c \frac{n}{2} \log \frac{n}{2} \right] + cn = cn(\log n - 1) + cn = cn \log n - cn + cn = cn \log n.$$

So our guess works, giving $T(n) = O(n \log n)$. You similarly prove the lower bound $T(n) = \Omega(n \log n)$, and therefore

$$\boxed{T(n) = \Theta(n \log n)}.$$

Easy way to remember substitution: *Guess* $\rightarrow$ *Substitute* $\rightarrow$ *Prove*.

Slide 30: Master method (theorem) recap

$$T(n) = a \cdot T(n/b) + f(n)$$

Idea: compare $f(n)$ (root level) with $n^{\log_b a}$ (leaf level).

- **Case 1:** $f(n) = O(n^{(\log_b a) - \epsilon})$ (polynomially smaller; leaf level dominates)

$$T(n) = \Theta(n^{\log_b a})$$

- **Case 2:** $f(n) = \Theta(n^{\log_b a})$ (all levels about the same)

$$T(n) = \Theta(n^{\log_b a} \log n)$$

- **Case 3:** $f(n) = \Omega(n^{(\log_b a) + \epsilon})$, and ... (root level dominates)

$$T(n) = \Theta(f(n))$$

Detailed explanation. The Master Theorem is a quick way to solve running times of divide-and-conquer algorithms that look like

$$T(n) = a T\left(\frac{n}{b}\right) + f(n).$$

What the parts mean:

- a : how many subproblems you create.
- n/b : each subproblem size (you shrink the input by factor b).
- $f(n)$: extra work you do outside the recursive calls (splitting, merging, etc.).

The key comparison. Compute this “benchmark”:

$$n^{\log_b a}$$

This represents the total work at the **bottom (leaves)** of the recursion tree. Then compare $f(n)$ to $n^{\log_b a}$. Three cases:

Case 1: $f(n)$ is smaller (leaves dominate). If

$$f(n) = O\left(n^{\log_b a - \epsilon}\right)$$

(meaning $f(n)$ is polynomially smaller), then

$$T(n) = \Theta\left(n^{\log_b a}\right).$$

Simple idea: most cost comes from doing many small subproblems.

Case 2: $f(n)$ is about the same size (all levels add up). If

$$f(n) = \Theta\left(n^{\log_b a}\right),$$

then

$$T(n) = \Theta\left(n^{\log_b a} \log n\right).$$

Simple idea: each level of recursion costs about the same, and there are $\log n$ levels.

Case 3: $f(n)$ is bigger (top/root dominates). If

$$f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$$

(and it satisfies a “regularity condition”, basically meaning it stays bigger consistently), then

$$T(n) = \Theta(f(n)).$$

Simple idea: the extra work $f(n)$ at the top overwhelms everything else.

Tiny examples:

- *Merge sort:* $T(n) = 2T(n/2) + n$. Here $n^{\log_2 2} = n$, which matches $f(n) = n \Rightarrow$ Case 2 $\Rightarrow T(n) = \Theta(n \log n)$.
- *Binary search:* $T(n) = T(n/2) + 1$. Here $n^{\log_2 1} = 1$, which matches $f(n) = 1 \Rightarrow$ Case 2 $\Rightarrow T(n) = \Theta(\log n)$.

Slide 31: Poll: practice with the master method

Slido poll

Solve the recurrence $T(n) = 4 \cdot T(n/2) + n \log n$. (multiple choice)

6 Summary

Slide 32: Review

(Small example: prices 10, 11, 7, 10, 6 over days 0–4, with the divide-and-conquer split diagram.)

- Brute force: $O(n^2)$ -time algorithm.
- Divide and Conquer: $O(n \log n)$ -time algorithm.
- Next: Dynamic programming — “smarter” D&C: dealing with subproblems better.
- Typically: optimal substructure + overlapping subproblems.